Azure Kubernetes Service Beginner Guide and Cheatsheet

Setting up Kubernetes manually requires managing complex master servers, but Azure Kubernetes Service handles that work for free. Learning AKS lets you focus purely on running your apps while Azure manages cluster health and updates.

What you'll learn:

1. Creating Your First AKS Cluster
2. Connecting Local Terminal to AKS
3. Setting Up Automatic Node Scaling
4. Checking Available Upgrades

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Creating Your First AKS Cluster

Creating a cluster manually takes hours, but Azure CLI lets you launch one in minutes. Azure manages the API server and control plane behind the scenes without charging extra maintenance fees. You only pay for the virtual machine worker nodes that run your actual app containers.

```
$az aks create --resource-group myResourceGroup --name myAKSCluster --node-count 2 --generate-ssh-key
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Connecting Local Terminal to AKS

To interact with your cloud cluster using kubectl, you must fetch its connection credentials first. The Azure CLI downloads these security keys directly into your local kubeconfig file automatically. Once connected, standard kubectl commands execute directly against your cloud nodes.

```
$az aks get-credentials --resource-group myResourceGroup --name myAKSCluster
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Setting Up Automatic Node Scaling

Manual scaling works fine for planned events, but automatic scaling prevents unexpected system outages. The Cluster Autoscaler watches for pods that cannot start due to missing hardware resources. When pods get stuck waiting, it provisions new worker nodes and removes them later when load drops.

```
$az aks update --resource-group myResourceGroup --name myAKSCluster --enable-cluster-autoscaler --min
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Checking Available Upgrades

Upgrading traditional Kubernetes clusters can break components if versions mismatch. AKS handles control plane upgrades safely and updates worker nodes one by one to prevent downtime. Always check available supported versions before triggering an upgrade command.

```
$az aks get-upgrades --resource-group myResourceGroup --name myAKSCluster --output table
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always use Azure RBAC integration for secure user access control.
+ Enable Cluster Autoscaler to avoid paying for idle virtual machines.
+ Keep node counts low (1-2 nodes) during initial testing to save money.
+ Group cluster resources inside a dedicated resource group for easy cleanup.
+ Use Azure Monitor to track cluster CPU and memory usage in real time.

Watch Out For

! Forgetting to delete test clusters, leading to unexpected cloud bills.
! Hardcoding passwords instead of using Managed Identities for Azure services.
! Upgrading cluster versions without testing app compatibility first.
! Forgetting to run 'az aks get-credentials', causing commands to hit the wrong cluster.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh